Ostbayerische Technische Hochschule Amberg-Weiden
Fakultät Elektrotechnik, Medien und Informatik

Studiengang Bachelor Künstliche Intelligenz

Bachelorarbeit

von

Sebastian **Weidner**

Imitation Learning unter Verwendung videokonditionierter Policies für visuell gesteuertes Roboterverhalten

Imitation Learning using Video-Conditioned Policies for Visually Guided Robotic Behavior

Ostbayerische Technische Hochschule Amberg-Weiden
Fakultät Elektrotechnik, Medien und Informatik

Studiengang Bachelor Künstliche Intelligenz

Bachelorarbeit

von

Sebastian **Weidner**

Imitation Learning unter Verwendung videokonditionierter Policies für visuell gesteuertes Roboterverhalten

Imitation Learning using Video-Conditioned Policies for Visually Guided Robotic Behavior

Bearbeitungszeitraum: von 20. Januar 2026
bis 20. Juni 2026

1. Prüfer: Prof. Dr.-Ing. Christian Bergler

2. Prüfer: Prof. Dr. phil. Tatyana Ivanovska

OTH Amberg-Weiden

Elektrotechnik, Medien und Informatik

Eigenständigkeitserklärung gemäß § 27 (8) ASPO

Name und Vorname der Studentin/des Studenten: **Weidner, Sebastian**

Studiengang: **Bachelor Künstliche Intelligenz**

Ich bestätige, dass ich die Bachelorarbeit mit dem Titel:

Imitation Learning unter Verwendung videokonditionierter Policies für visuell gesteuertes Roboterverhalten

selbständig verfasst, noch nicht anderweitig für Prüfungszwecke vorgelegt, keine anderen als die angegebenen Quellen oder Hilfsmittel benutzt sowie wörtliche und sinngemäße Zitate als solche gekennzeichnet habe.

Datum: June 13, 2026

Unterschrift:

OTH Amberg-Weiden

Elektrotechnik, Medien und Informatik

Bachelorarbeit Zusammenfassung

Studentin/Student (Name, Vorname):	**Weidner, Sebastian**
Studiengang:	Bachelor Künstliche Intelligenz
Aufgabensteller, Professor:	Prof. Dr.-Ing. Christian Bergler
Durchgeführt in (Firma/Behörde/Hochschule):	OTH Amberg-Weiden
Betreuer in Firma/Behörde:	Prof. Dr.-Ing. Christian Bergler
Ausgabedatum: 20. Januar 2026	Abgabedatum: 20. Juni 2026

Titel:

Imitation Learning unter Verwendung videokonditionierter Policies für visuell gesteuertes Roboterverhalten

---

Zusammenfassung:

Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet clita kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit amet. Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet clita kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit amet.

Schlüsselwörter: LaTeX, Textsatz, Formeln, Graphiken

Abstract:

Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet clita kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit amet. Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet clita kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit amet.

Keywords: LaTeX, Textsatz, Formeln, Graphiken

Contents

Acronyms

Symbole, Formelzeichen und Einheiten

Chapter 1

Foundations

1.1 Deep Learning

1.1.1 Neurons

1.1.2 Neural Networks

1.1.3 Neural Networks

Artificial Neural Networks (ANNs) are computational models inspired by the structure and functionality of biological brain. They are designed to learn complex relationships from data and have become a fundamental component of modern machine learning. ANNs are capable of approximating highly nonlinear functions and are therefore widely used in applications such as image recognition, natural language processing, forecasting, and control systems.

Neuron

The basic element of an artificial neural network is the artificial neuron. The concept is motivated by biological neurons in the human brain, which communicate through electrochemical signals. In a simplified model, a biological neuron receives signals from other neurons through dendrites, processes the received information, and transmits an output signal through its axon. Artificial neurons mimic this behavior mathematically by combining several input values into a single output.

History. The first mathematical model of an artificial neuron was proposed by McCulloch and Pitts in 1943 [**McCulloch1943ALN**]. Their model consisted of a binary threshold unit that produced an output only when the weighted sum of its inputs exceeded a predefined threshold. Although this model did not include a learning mechanism, it established the foundation for later neural network research.

A major extension of this idea was introduced by Rosenblatt in 1958 with the perceptron [**Rosenblatt1958ThePerceptron**]. In contrast to the McCulloch–Pitts neuron, the perceptron introduced trainable weights and a learning rule, making it possible

to adapt the model parameters directly from data. However, the perceptron still relied on a threshold-based activation and was therefore limited to linearly separable classification problems [**Minsky1969Perceptrons**].

This limitation was highlighted by Minsky and Papert, who showed that a single perceptron cannot represent functions such as XOR [**Minsky1969Perceptrons**]. The development of multilayer neural networks together with differentiable activation functions later overcame this restriction. These advances made it possible to train networks with multiple layers and enabled the modern neural network architectures that are used in deep learning today.

Mathematical Formulation. A neuron receives an input vector $\mathbf{x} = [x_1, x_2, \ldots, x_n]^T$, where each vector element x_i is associated with a trainable weight w_i from the corresponding weight vector $\mathbf{w} = [w_1, w_2, \ldots, w_n]^T$. The neuron first computes a weighted sum of its inputs and adds a bias term b. The resulting pre-activation value is given by:

$$z = \sum_{i=1}^{n} w_i x_i + b \tag{1.1}$$

The value z represents a linear combination of the input features. To enable the modeling of non-linear relationships, an activation function $\phi(\cdot)$ is subsequently applied, yielding the neuron's output:

$$y = \phi(z) = \phi\left(\sum_{i=1}^{n} w_i x_i + b\right) \tag{1.2}$$

Without the activation function, a composition of multiple neurons would still correspond to a single linear transformation and therefore could not represent complex non-linear patterns. Common activation functions used in modern neural networks include the sigmoid function, the hyperbolic tangent (tanh), and the Rectified Linear Unit (ReLU).

The weighted sum in Equation 1.1 can be expressed more compactly using vector notation and ending in the form of:

$$y = \phi\Big(\mathbf{w}^T\mathbf{x} + b\Big) = \phi\left(\begin{bmatrix} w_1 & w_2 & \cdots & w_n \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} + b\right) \tag{1.3}$$

This vector formulation serves as the basis for the matrix representation of an entire neural network layer, where multiple neurons are evaluated simultaneously.

Neural Networks

A neural network is a collection of interconnected neurons organized into layers. The most common form of such architectures is the *Multilayer Perceptron* (MLP), also

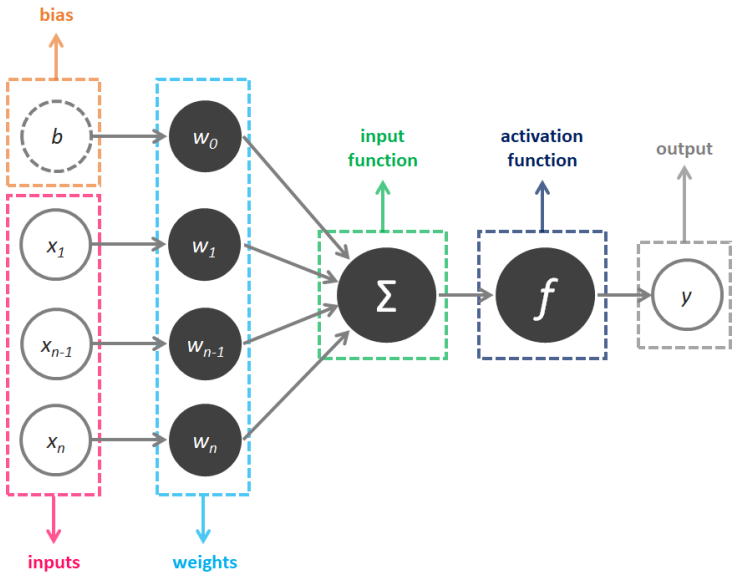

Figure 1.1: Artificial Neuron. Image source: [**xxx**].

referred to as a feedforward neural network, where information flows in one direction from the input layer through one or more hidden layers to the output layer (without forming cycles). The parameters of the network, including the weights and biases, are learned from data using optimization algorithms such as gradient descent. The ability of neural networks to learn complex functions from data has made them a powerful tool for a wide range of applications in machine learning and artificial intelligence.

A neural network consists of multiple interconnected neurons organized into layers The input layer receives the feature vector, while the output layer produces the final prediction. The hidden layers perform intermediate computations that allow the network to learn increasingly abstract representations of the input data.

In a *Multilayer Perceptron Network*, each neuron in a layer receives the outputs of the neurons in the preceding layer as inputs. In a fully connected layer, every neuron is connected to every neuron in the subsequent layer. The output of a layer therefore serves as the input to the next layer.

The computation of an entire layer can be expressed compactly using matrix notation. Let $\mathbf{x}^{(l-1)}$ denote the output vector of the previous layer, $\mathbf{W}^{(l)}$ the weight matrix, and $\mathbf{b}^{(l)}$ the bias vector of layer l. M are the number of neurons in layer l and N the number of neurons in the previous layer $l-1$. The pre-activation vector is given by:

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)}\mathbf{x}^{(l-1)} + \mathbf{b}^{(l)}, \tag{1.4}$$

which can be written explicitly as:

$$\begin{bmatrix} w_{11} & \cdots & w_{1N} \\ w_{21} & \cdots & w_{2N} \\ \vdots & \ddots & \vdots \\ w_{M1} & \cdots & w_{MN} \end{bmatrix}^{(l)} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{bmatrix}^{(l-1)} + \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_M \end{bmatrix}^{(l)} = \begin{bmatrix} w_{11}x_1 + \cdots + w_{1N}x_N + b_1 \\ w_{21}x_1 + \cdots + w_{2N}x_N + b_2 \\ \vdots \\ w_{M1}x_1 + \cdots + w_{MN}x_N + b_M \end{bmatrix} \tag{1.5}$$

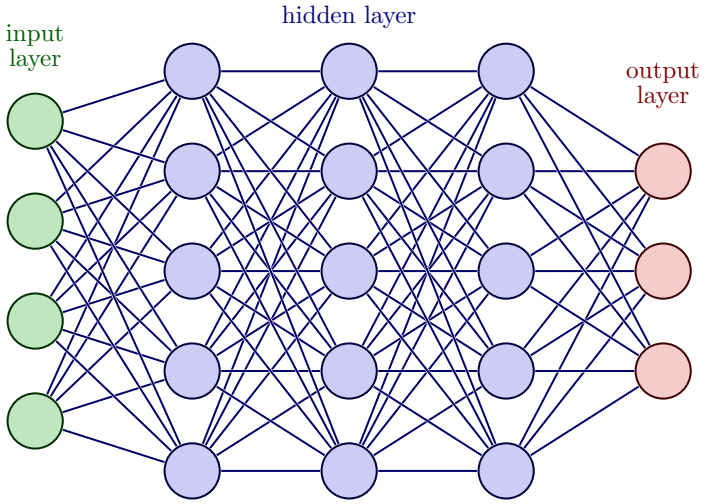

Figure 1.2: Neural Network Architecture. Image source: [**xxx**].

In other words, each neuron in layer l computes its own weighted sum over all outputs of the previous layer and then adds its corresponding bias term.

Equivalently, the bias can be absorbed into the matrix multiplication by augmenting the input vector with a constant entry 1 and extending the weight matrix by one additional column:

$$\begin{bmatrix} z_1^{(l)} \\ z_2^{(l)} \\ \vdots \\ z_M^{(l)} \end{bmatrix} = \begin{bmatrix} w_{11}^{(l)} & \cdots & w_{1N}^{(l)} & b_1^{(l)} \\ w_{21}^{(l)} & \cdots & w_{2N}^{(l)} & b_2^{(l)} \\ \vdots & \ddots & \vdots & \vdots \\ w_{M1}^{(l)} & \cdots & w_{MN}^{(l)} & b_M^{(l)} \end{bmatrix} \begin{bmatrix} x_1^{(l-1)} \\ x_2^{(l-1)} \\ \vdots \\ x_N^{(l-1)} \\ 1 \end{bmatrix} \tag{1.6}$$

Then the activation function is applied element-wise to the resulting pre-activation vector $\mathbf{z}^{(l)}$ to produce the output of layer l:

$$\mathbf{x}^{(l)} = \phi(\mathbf{z}^{(l)}) = \phi\left(\mathbf{W}^{(l)}\mathbf{x}^{(l-1)} + \mathbf{b}^{(l)}\right). \tag{1.7}$$

The

Non-Linear Activation Functions in Neural Networks

The introduction of non-linear activation functions is essential, as a composition of purely linear transformations would again result in a linear model. By combining multiple layers with non-linear activations, neural networks can approximate highly complex functions and learn non-linear, intricate patterns from data. In fact, sufficiently large neural networks are universal function approximators, meaning they can approximate a wide range of continuous functions to arbitrary accuracy. Common activation functions used in modern neural networks include the functions GELU

(Gaussian Error Linear Unit), Rectified Linear Unit (ReLU), Leaky ReLU, the tanh (hyperbolic tangent) and the Swish function. But there exist many more. Each of these functions has its own characteristics and can result in different learning dynamics and performance.

Prediction

The output layer is responsible for producing the network's final prediction. Its activation function is chosen based on the task:

- **Regression:** For regression problems, a linear activation directly outputs a continuous value.
- **Binary Classification:** For binary classification (true/false), a single neuron with a sigmoid function outputs a probability between 0 and 1.
- **Multi-Class Classification:** For multi-class classification, a softmax layer produces a probability distribution over the classes, where each output neuron corresponds to a specific class and the softmax function ensures that the outputs sum to 1, representing the predicted probabilities for each class.

The choice of activation function in the output layer is crucial, as it directly affects the interpretation of the network's predictions and the appropriate loss function to use during training.

Training Neural Networks

The parameters of a neural network, namely the weights and biases, are learned from data during training. This is typically achieved by minimizing a loss function using gradient-based optimization methods such as gradient descent and its variants. The gradients required for optimization are efficiently computed using the backpropagation algorithm, which propagates the prediction error from the output layer back through the network. This learning process enables neural networks to adapt their internal representations and achieve strong performance across a wide range of machine learning tasks.

What is a Gradient? The gradient is a fundamental concept in optimization and machine learning. It describes how a function changes with respect to its input variables and points in the direction of the steepest increase of the function. In the special case of a function with only one variable, the gradient reduces to the first derivative, which corresponds to the slope of the function at a given point. In simple terms, the gradient indicates how the function value changes when making a small step to the right or left away from a specific point.

To better understand the concept of gradients, consider Figure 1.3. We want to determine the slope of the blue curve at the point $x = 1$. To do this, we calculate the gradient at that point. First, we compute the derivative $f'(x) = 2x$ of the function $f(x) = x^2 - 3$ and evaluate it at $x = 1$. The first derivative $f'(x)$ provides the slope at any point on the function $f(x)$. In this example, the slope at $x = 1$ is $f'(1) = 2$. To

verify this result, the slope triangle is shown in **??**. It provides a visual representation of the slope and confirms the calculated value of with the slope $m = 2$.

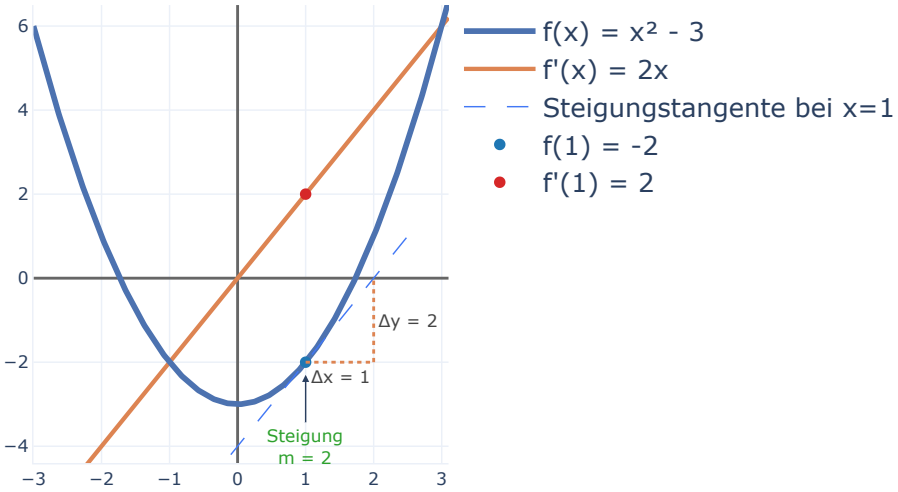

Figure 1.3: Neural Network Architecture. Image source: [**xxx**].

In general, the gradient indicates whether a function is increasing or decreasing at a specific point and how steeply it changes. A positive gradient means that the function is increasing, while a negative gradient indicates that the function is decreasing. The magnitude of the gradient describes how steeply the function changes. Larger magnitudes correspond to steeper slopes, while values close to zero indicate relatively flat regions.

Gradient Descent. The basic principle of gradient descent can be understood by considering a simplified setting with a single parameter—a network weight—as illustrated in the left part of Figure **??**. The horizontal axis represents the value of the weight, and the vertical axis shows the resulting cast or known as loss, which comes from the difference between the predicted and true values in the training data. The resulting curve is the loss landscape for that one parameter. At any point on this curve, the derivative (or gradient) indicates the slope at that specific point on the curve. The current network weight, shown by the black dot must be updated in a way, to reduce the loss. Gradient descent consists of two main steps:

1. **Calculate the gradient:** The first step is to compute the gradient of the loss function with respect to the weight. This gradient indicates how the loss changes as the weight is adjusted.

2. **Update the weight:** The weight is then updated by moving in the opposite direction of the gradient. This is because the gradient points in the direction of

steepest ascent, so moving in the opposite direction will lead to a decrease in the loss. The update can be expressed mathematically as:

$$W_{t+1} = W_t - \alpha \cdot \nabla L(W_t) \tag{1.8}$$

where α is the learning rate, a hyperparameter that controls the step size of the update, and $\nabla L(W_t)$ is the gradient of the loss function with respect to the weight at its current value.

In our example, the gradient is positive, which means that increasing the weight would increase the loss. Therefore, we need to move the weight in the opposite direction (to the left) to reduce the loss. This process is repeated iteratively, with the weight being updated in each iteration until convergence is achieved, meaning that the loss is minimized and the network's predictions are sufficiently accurate, see the black arrows in Figure **??**.

First the gradient is calculated. If the derivative is positive (the tangent slopes upward to the right), increasing the weight would increase the cost; conversely, moving the weight in the opposite direction (to the left) reduces the cost.

Gradient descent formalizes this intuition by always updating the parameter in the direction opposite to the gradient – that is, downhill along the loss surface.

The training of a neural network involves adjusting its parameters (weights and biases) to minimize a loss function. The loss function $\mathcal{L}$ is a measure of how well the network is able to predict the target value and quantifies the difference between the network's predictions and the true labels in the training data. This is typically done using optimization algorithms such as gradient descent, which is the most common optimization algorithm in practice.

To better understand the gradient descent algorithm, consider a Gradient descent iteratively updates the parameters in the direction that reduces the loss.

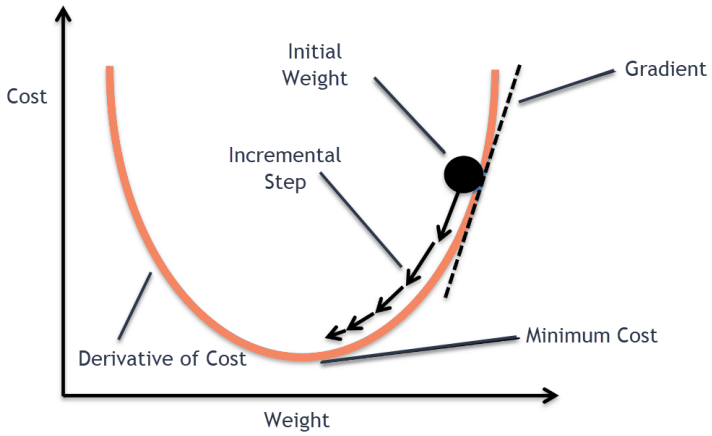

Figure 1.4: Neural Network Architecture. Image source: [**xxx**].

Gradient descent is the most common optimization algorithm used in practice. The gradients needed for these updates are computed using the backpropagation algorithm, which efficiently propagates the error from the output layer back through the

network to compute the necessary gradients for all parameters. For each neuron, the corresponding proportion of the total error is calculated and the weights are adjusted accordingly. Through this process, neural networks can learn complex patterns and achieve high performance on a variety of tasks.

. If the predicted output is close to the target, the value of the loss function will be small. If the predicted output is far from the target, the value of the loss function will be large. The goal of the training process is to minimize the loss function by adjusting the weights of the network. The loss function can be any differentiable function, but common choices are Mean Squared Error (MSE) for regression tasks and cross-entropy for classification tasks. The MSE loss function is defined as: Where yi is the target value for the i-th sample, yi is the predicted value for the i-th sample, and N is the number of samples. The cross-entropy loss function is defined as:

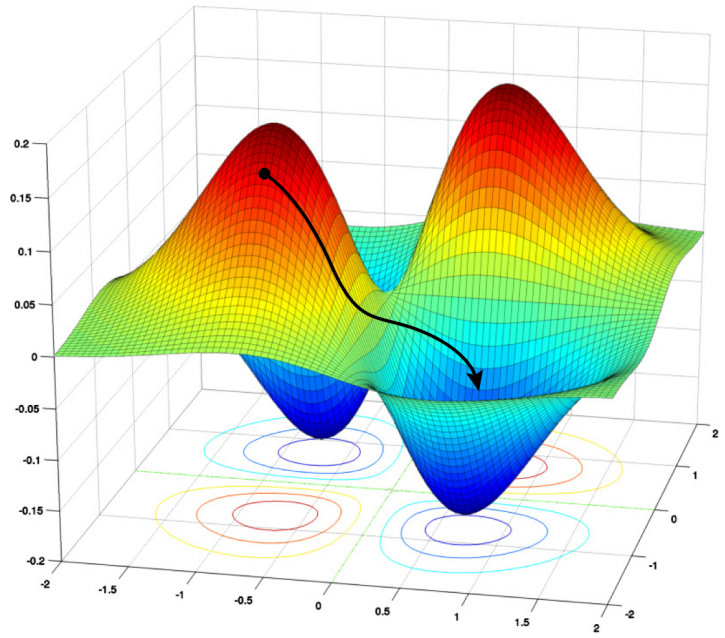

Figure 1.5: Neural Network Architecture. Image source: [**xxx**].

—

List of Figures

List of Tables